DeskMate——基于全新 BK7258 平台的 openvela AI 桌面助手

2026 首届 openvela AI 硬件开发者大赛 · 技术报告（队伍 272 · token 无极限）

1、信息表

项目	内容
作品名称	DeskMate——基于全新 BK7258 平台的 openvela AI 桌面助手
队伍名称	token 无极限（队伍编号 272）
团队分工	陈泽沛 C-Luv9615（队长）：平台移植、WiFi 适配；应用：设计 acp bridge 远程连接 coding agent. 张怡 JUSTiceMaMo：Audio 输入输出，应用层：MCP 设计，ai_agent GUI； 郭胜元 suoyuan666：LCD 显示，SMP 双核，真机稳定性
选题方向	新硬件平台适配（主）+ AI 硬件产品创新（辅）

2、摘要

本作品将 openvela 移植到 BK7258 该 SoC：多核启动、外设驱动、Mailbox RPMsg、SPI 点亮 GC9D01 圆屏，并打通 Wi-Fi 全链路。AI 侧深度使用 openvela ai_agent：在 fork 上实现 MQTT 远程 Agent 控制与 MCP 接入常用研发平台（Gerrit、Jira），在 QEMU 跑通语音驱动的 Gerrit 编码闭环；并以 DeskMate Link 协议把权限决策放到设备端。3 名成员、168 个 AI 会话完成开发。

3、正文

3.1 绪论

项目背景与问题定义：openvela 的落地广度取决于芯片覆盖。BK7258 是 Beken 三核 Wi-Fi SoC（16 MiB PSRAM）：CPU0 为 CP 固件核，CPU1 加 CPU2 在参考设计中组成 AP 双核 SMP；本作品当前 AP 跑在 CPU1、CPU2 未启用（原因见 3.5）。官方只提供 FreeRTOS（Armino）SDK，闭源 Wi-Fi 协议栈 libwifi.a 深度绑定 FreeRTOS 的假设，此前没有 NuttX 移植可参考。

AI 侧的背景是桌面助手场景。Claude Code 一类 Agent 已能代替人执行真实操作，但"授权 AI 动真实环境"的确认环节仍锁在 PC 屏幕上；桌面实体设备需要一个可信的权限入口。openvela 自带的 ai_agent 提供了设备端 Agent 框架（ReAct 循环、MCP 客户端、语音通道、LVGL 界面），但它与远程 Agent 的协作和权限交互此前没有现成做法。

技术难点：难点分布在从构建、启动、跨核、驱动到协议栈的每一层，复盘文档按失真层记录了 22 项根因。

① 从零的构建与启动：CP/AP 双镜像各自的链接脚本与 L2 打包（all-app.bin）、PSRAM 堆初始化（16 MiB）、PM 初始化时序（曾把 PSRAM 所在电源域关掉）、从核释放时序（halt/pwr_dw 位检查）、链接脚本孤儿段（.dtcm_sec_data 落在初始化区间外）、Wi-Fi 数据分区接入。

② AMP 跨核系统：CP 与 AP 是两份独立镜像，需要冻结共享 SRAM 契约、把 Mailbox v2（IRQ79）限定为 doorbell 并在其上实现 RPTUN/RPMsg VirtIO、AP 经 RPMsg UART 提供控制台、支持 AMP 重启后的 RPTUN 重连；整个 bring-up 按 A0（CPU1 复位桩）到 A4（外设迁移）分阶段关卡推进。

③ 外设驱动：GPIO AON 配置位布局修正、UART ISR 按使能位分发、硬中断内 printf 触发 assert（改用 syslog）、SPI 两条路径（bit-bang 与硬件 SPI1）、LCD 电源轨（LDO_3V3、背光、复位，缺一个屏就不亮）、GDMA0、Timer0 下半层。

④ 闭源 Wi-Fi 库的黑盒契约：新分配内存默认清零、中断屏蔽寄存器返回值语义、dcache flush 义务、校准链调用顺序与 libm 依赖；这类问题编译链接全部干净、只在运行时失败。

⑤ 驱动与 NuttX 协议栈的交接：收帧在 IOB 中的位置（L3 载荷负偏移）、IOB 链与配额归还、EAPOL 分流义务、netdev 协议栈侧字段（d_mac、carrier、vif_idx）。

⑥ 可复现的验证链：自动 reboot-to-bootloader、交互式 GetBus 烧录、打包解码门禁、源文件 mtime 对比打包时刻证明固件一致性；没有这条链，前面每一项的迭代都推不动。

⑦ 应用层：把权限决策做成确定性协议，LLM 输出不能成为权限来源。

创新点：

① 新板子的从零 bring-up：BK7258 是 openvela 首次落地的全新平台，三核 Wi-Fi SoC 从链接脚本、双镜像打包、跨核通信到 Wi-Fi 全链路均无先例可抄，对应大赛重点鼓励的新硬件平台适配方向。

② 不使用芯片原厂 SDK 运行时：Beken 官方只提供 Armino/FreeRTOS SDK，本作品把运行时整体换成 openvela（NuttX），Armino 仅作契约参考而非运行底座。闭源 libwifi.a 深度绑定 FreeRTOS 假设，其行为契约只能以 Armino 源码为说明书黑盒还原：OSAL 语义翻译（中断屏蔽返回值、内存清零前提、dcache 维护）、任务/队列/内存的原语映射、初始化链顺序。初期手写 shim 顶替换来静默偏差，最终以 cmp 校验的 Armino 底本加 NuttX 集成层完成适配，难点在判断每处差异是集成点还是契约本体，判错一边要么编译不过、要么运行时静默出错。

③ 应用形态创新：把「AI 权限确认」从 PC 屏幕移到桌面实体设备。设备端 rask 一句话即可驱动远端 Claude Code，LLM 推理中也能经 remote_agent_prompt 工具自主发起远程回合；权限决策是确定性协议（仅当前 prompt.id 一次、本地闭锁加桥端终判、LLM 输出无权限决定权，对提示注入是结构性防御），并把会话计费（charge/tokens 双槽）实时回传设备显示。同一模型在 DeskMate Link 与 ai_agent remote_ctrl 两条路径各实现一次，覆盖自研应用与框架集成两种形态。

3.2 系统方案设计

系统总体架构：自下而上四层。硬件层：BK7258 三核 SoC，CPU0（CP）运行 Wi-Fi 固件，CPU1（AP）运行 openvela 与应用（CPU2 暂未启用），CP 与 AP 经 Mailbox v2 加 RPTUN/RPMsg 通信。网络层：Wi-Fi STA 接入家庭路由器，NuttX 网络栈。消息层：局域网 MQTT broker，主题 deskmate/<device-id>/device-to-bridge 与 bridge-to-device。PC 端：ACP 桥，后端为 mimo acp 或 kiro-cli。设备侧有两条 AI 路径：自研 deskmate 客户端（app/deskmate），以及 openvela ai_agent（VelaClaw）内的 remote_ctrl 通道。

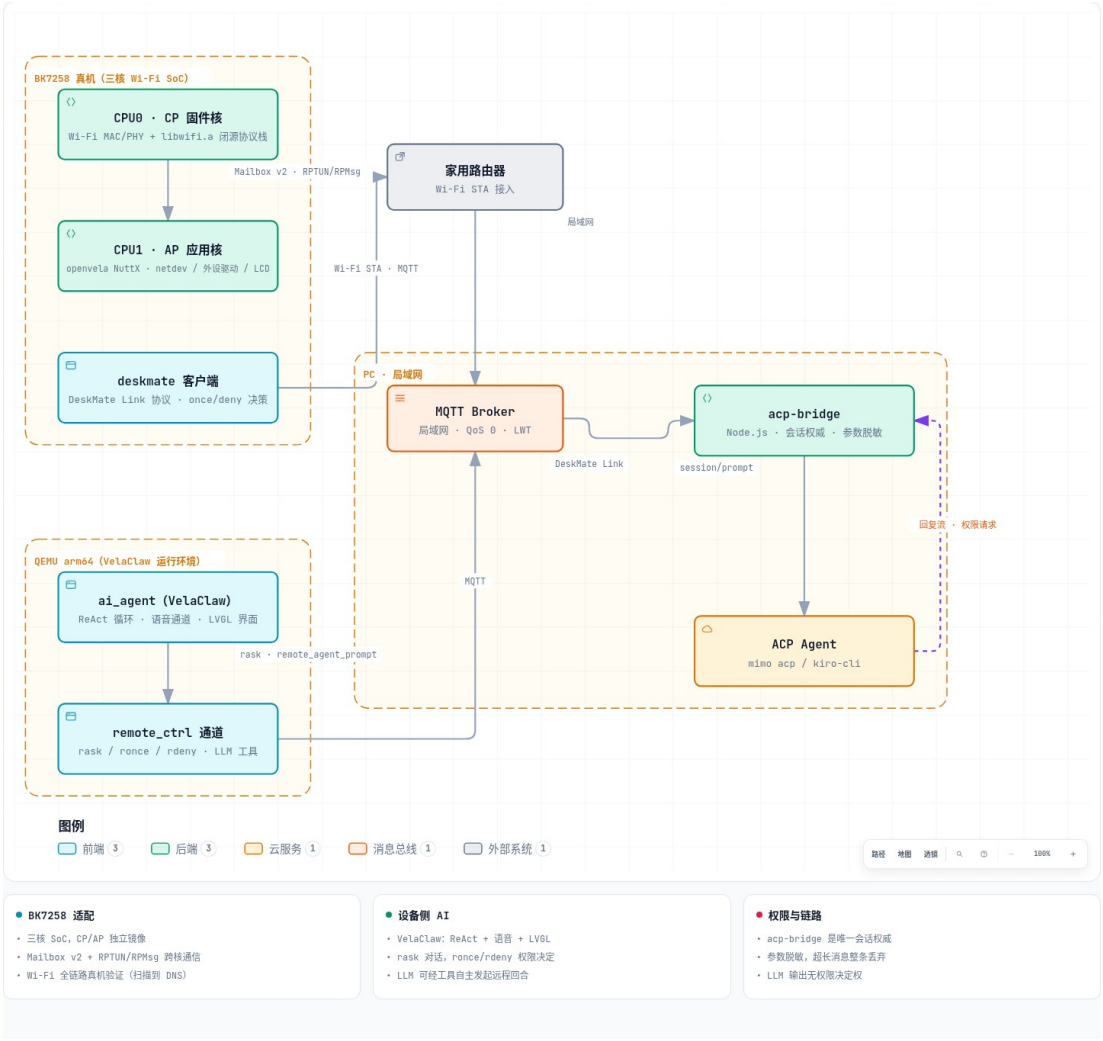


图 1：DeskMate 系统总体架构

方案论证与选型：开发板：BK7258 没有现成 openvela 移植，多核、Wi-Fi、16 MiB PSRAM 适配 AI 交互硬件形态，移植难度也最能体现新硬件适配方向的价值。端/云职责：端侧承担协议状态机、权限闭锁、输入与显示，不跑大模型；PC 侧运行 LLM Agent，由 ACP 桥产生对话与工具调用。断网与弱网降级：MQTT 断连自动重连，每次重连生成新连接 nonce 重新握手，收到匹配的权威快照之前保持 Offline、不消费任何旧决策；QoS 0 序列缺口只记录、不改变权威状态。备选方案对比：USB CDC（换行分帧）作为第二传输层保留在抽象里；优先 MQTT，因为它保序并保留消息边界，且 Wi-Fi 联网本身就是本次适配的核心验证目标。

关键模块设计：

① DeskMate Link 协议 v1（已冻结）：hello/hello_ack 能力协商、权威快照（nonce/epoch 锁定会话）、decision（once/deny）、usage_snapshot 计费遥测（独立严格递增 usage_seq）、对话轮次（设备侧由 rask 命令或 LLM 工具发起，回复流式回传）；消息上限 4096 UTF-8 字节，超长整条丢弃、绝不截断。

② 权限决策模块：controller 对当前精确 prompt.id 维持本地决策闭锁，成功发送后同一

prompt.id 的重复输入被拒绝，直到更新的权威快照清除或替换该 prompt；桥端是最终消费者，拒绝一切重复、过期、不匹配的决策。

③ ai_agent remote_ctrl 通道（fork 贡献）：rask/rstat/rreply/rmodel/rusage/ronce/rdeny 控制台命令，remote_agent_prompt 与 remote_agent_status 两个 LLM 工具；刻意不提供审批工具，权限决定只能由人在控制台敲 ronce/rdeny。rask 的回复与待审批提示镜像到聊天界面（控制台卡片、LVGL 聊天视图、语音管线）；remote_agent_prompt 的同步回合不镜像，结果由本地模型整合进自己的回答。

④ 传输抽象层：transport.h 统一接口，NuttX MQTT-C 为首个实现，USB CDC 预留。

3.3 核心算法与技术原理

AI 算法实现：端侧两条路径。其一，openvela ai_agent（VelaClaw）的 ReAct agent loop，带 MCP 客户端、语音通道（Volcengine ASR v3）与 LVGL 界面，运行在 QEMU arm64（goldfish-arm64-v8a-ap defconfig 加配置补丁）。其二，自研 deskmate 客户端，不跑模型，只承载协议状态机与权限决策。云端模型：PC 桥后端为 mimo acp（大赛 MiMo Token）与 kiro-cli，经 ACP 协议产生对话与工具调用；模型选择可经 models 消息在会话内原位切换（保留历史）。计费遥测经 usage_snapshot 回传设备，显示为 Billing: <charge> / <tokens> 两槽；客户端不从 token 数推算价格，也不把上下文窗口占用当计费。

关键机制设计：① 确定性权限状态机：唯一能产生 once/deny 的是当前精确 prompt.id，且每个 prompt.id 至多一次；重复、过期、未知、断连后的决策一律不得触发 PC 动作；LLM 输出不得做权限决定（对提示注入的结构性防御）。② 防重放与乱序：连接 nonce 加 epoch 权威快照锁定会话，usage_seq 严格递增，QoS 0 缺口只记账。③ 驱动层关键机制：中断分发按使能位过滤（清除仍用未掩码 raw，防止未使能位 latch 中断线）、dcache Clean+Invalidate、ke_malloc 与 os_malloc_wifi_buffer 清零补齐闭源库前提。④ Gerrit 编码闭环：QEMU 内 agent 经 MCP 调用 rc_read_file/rc_write_patch/rc_run/rc_git，同一 Change-Id 用 git commit --amend 推送新 patchset，patchset 号向 Gerrit 服务器实时查询、由 agent 播报权威值。

openvela 系统能力的深度运用：图形、AI、多媒体三项均落地。图形：BK7258 SPI LCD（JD9853 2 寸 240x296、GC9D01 0.71 寸 160x160）加 VelaClaw 的 LVGL 界面（静态中文字体，中文渲染可用）。AI：ai_agent 框架的深度使用与 fork 贡献，ACP 桥接 MiMo。多媒体：VelaClaw 语音通道（Volcengine ASR v3），语音输入闭环已在 QEMU 验证。用到的 openvela/NuttX 组件：netdev 网络设备框架、MQTT-C、DHCP/DNS 客户端、RPTUN/RPMsg VirtIO、Mailbox v2、IOB 缓冲框架、cJSON。改进建议两条：NETDEV_STATISTICS 需要驱动显式 select 才可用，建议对 lowerhalf 驱动默认可见；跨 IOB 链大帧路径存在静默丢弃风险（NET_ETH_PKTSIZE 超过 626 会重现），建议上游补齐或写入文档。

3.4 系统实现

软件 / 固件架构：分五块。chips/bk7258/ 芯片支持层（约 550 个 .c 文件）：以 Armino 源码为契约参考底本（cmp 校验），工程主体是 NuttX 集成层——bk7258_start.c 多核启动、OSAL adapter 表（中断/任务/队列/内存原语翻译到 NuttX）、flash.ld 与 flash_ap.ld 链接脚本、GPIO/UART/TRNG/PM/SARADC/GDMA/SPI 各外设的下半层驱动与 AON 位布局修正。board/bk7258-devkit/ 板级层：cp 全功能镜像（Wi-Fi、LCD、usrsock 服务端）、ap 应用核镜像（usrsock 客户端，跨核上网）、psram 隔离调试等多份配置，L2 打包脚本、多核 bring-up 研究记录（1021 行）。app/deskmate/ 设备端参考客户端：可移植协议状态核心、会话控制器、MQTT 传输、控制台适配，附主机端 gcc 单元测试。tools/acp-bridge/（Node.js）PC 桥：ACP 适配器、会话权威、MQTT v5 传输、用量记账、参数脱敏、fake 后端确定性回路测试。packages/ai_agent fork（github.com/C-Luv9615/packages_ai_agent，dev-ai-contest-2026 分支）：MQTT remote_ctrl 通道与控制台命令、remote_agent_prompt/status LLM 工具。

数据流与关键流程设计：上行：Wi-Fi MAC 中断进入 adapter（ke 调度器加闭源固件），经 rx_submit 交给 NuttX netdev（netpkt 分配、以太头负偏移处理），再经 IPv4/UDP 到 MQTT-C，经 broker 到 PC 桥，最后到 mimo acp 或 kiro-cli。下行：Agent 输出流式回到设备，写入 8 KiB 转录缓存并打印控制台；权限请求以权威快照下发，once/deny 决策原路返回。

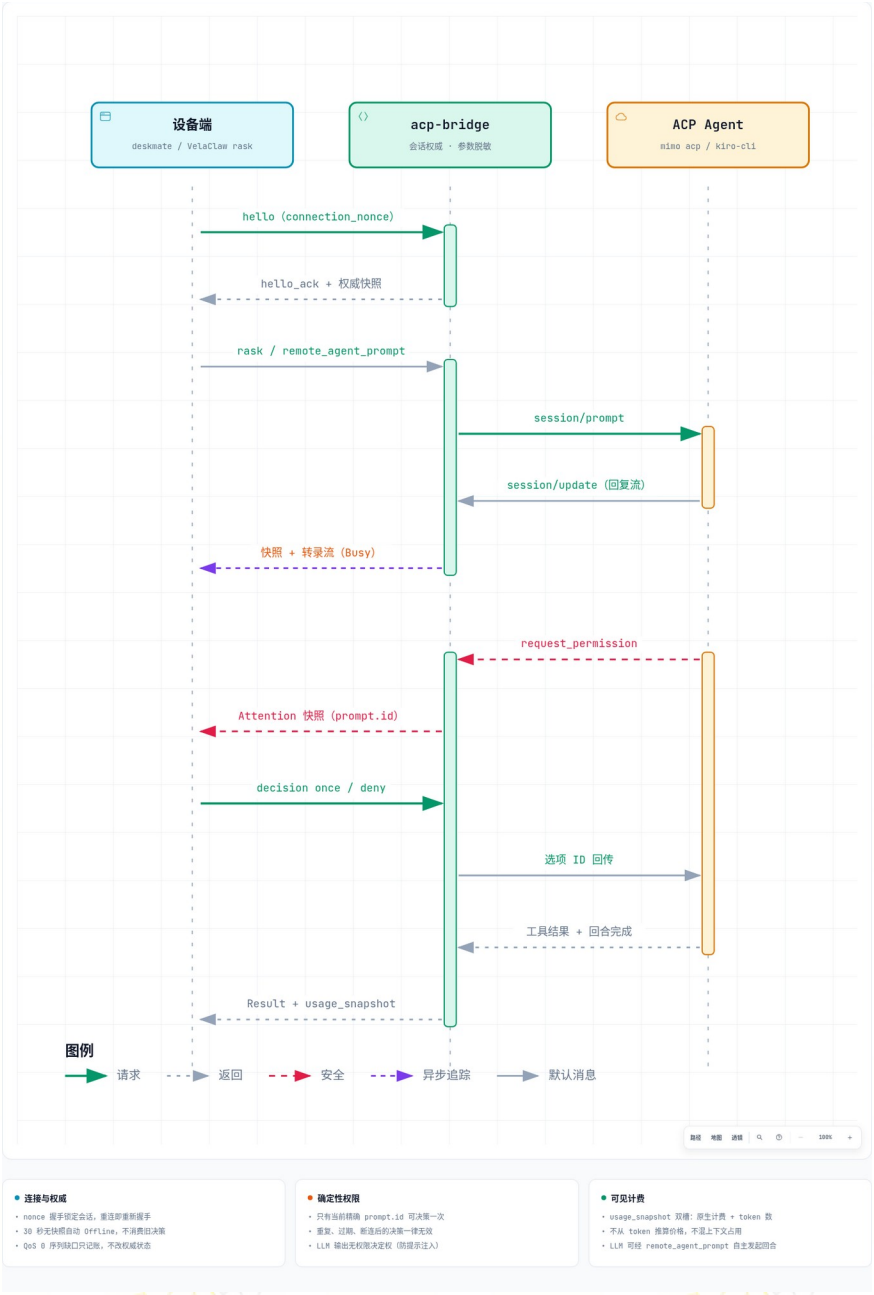


图 2：DeskMate 数据流与权限决策流程

硬件设计与适配：完成全新硬件平台适配（是）。芯片/开发板：Beken BK7258（三核 Wi-Fi SoC，16 MiB PSRAM；CPU0 为 CP 固件核、CPU1 为 AP 应用核、CPU2 未启用），UART0 控制台。驱动类型：UART、GPIO（含 AON 配置位布局修正）、TRNG、PM、SARADC、GDMA、Timer0 下半层、SPI（bit-bang 与 SPI1 硬件两种）、跨核 Mailbox v2 加 RPTUN/RPMsg（IRQ79 门铃、共享 SRAM、RPMsg UART 控制台）。

显示：JD9853 与 GC9D01 两款 SPI TFT 的下半层驱动，板级胶水含 LDO_3V3 使能（GPIO52）、背光（GPIO25）、复位（GPIO45）；GC9D01 已在真机点亮（初始化、背光、渲染），JD9853 驱动与 CP 配置就绪。

Wi-Fi：闭源库经 adapter 集成。适配难点与解决方案（代表性五条，完整 22 条及判据见仓内

复盘文档 docs/bk7258-network-bringup-retrospective.md）：

① 中断屏蔽寄存器返回值契约反转：authority 约定 disable 变体返回清位前的使能寄存器值以供精确恢复，原移植返回 0，MAC 中断被永久屏蔽（扫描全部 10 秒超时）；修为 spinlock 下读-改-写并返回改前值。

② EAPOL 未分流，四次握手不开始；在 netif 路径前分流给 ke_l2_packet_tx()。

③ 收帧在 IOB 中的位置偏一个以太头：NuttX netdev 要求 IOB_DATA 指向 L3 载荷；改用 netpkt_alloc()/netpkt_copyin() 让框架处理负偏移。

④ PM 初始化关闭 PSRAM 所在电源域，16 MiB 活跃堆失效；PM 硬件初始化前移到 nx_start() 之前。

⑤ 链接脚本孤儿段 .dtcm_sec_data 不初始化；折进 .data。

打包与烧录：CP/AP 双镜像 L2 all-app.bin 完整打包；自动 reboot-to-bootloader 加交互式 GetBus 烧录流程由自研 Skill bk7258-worktree-build 支撑，烧写判据 Writing Flash OK，loader 日志留存于 package 目录。

应用 / 交互端设计：设备端的主要入口在 VelaClaw（ai_agent fork）：rask 发起远程对话，回复进入控制台卡片、LVGL 聊天视图与语音管线；待审批请求弹 toast 提示，人在控制台敲 ronce/rdeny 决定；rmodel 切换模型、rusage 查用量、rreply 读会话转录；remote_agent_prompt 与 remote_agent_status 注册为 LLM 工具，agent 推理中可自主发起远程回合（同步、上限 120 秒），结果由本地模型整合进回答。deskmate 前台命令（app/deskmate，BK7258 上的 NuttX 参考实现）：once/deny 权限键与 usage 计费查询走同一决策路径。PC 端 acp-bridge 提供 REPL 与终端客户端。LCD 与按键 UI 列入后续计划。

自定义 Skill：仓内沉淀 .claude/skills/bk7258-worktree-build：覆盖 worktree 映射、构建、打包解码门禁、自动重启进 bootloader 的交互烧录（含可见手动复位提示），支撑全部真机迭代。开发中使用官方 nuttx-driver-development、driver-code-reviewer、kconfig-tweak。本作品的运行时 Agent 能力经 ai_agent 的 MCP 工具（rc_*）接入，未定义 /data/agent/skills/ 目录的运行时 Skill；"至少沉淀 1 个 Skill"由上述开发期 Skill 满足。

3.5 系统测试与结果分析（务必提供量化数据）

测试环境：BK7258 真机（UART0 控制台、USB 串口烧录，openvela 工作区 build.sh 构建 cp + ap 双镜像，AP 侧含 usrsock 跨核网络，默认构建即含已交付的网络功能）；QEMU arm64（goldfish/qemu-arm64-v8a-ap，VelaClaw 运行环境）；PC 桥（Node.js，后端 mimo acp 或 kiro-cli）。验证用 AP 为手机热点（Luv）。证据留存：仓内 evidence-20260908 至 evidence-20260920 六批 UART 日志（PSK 均已脱敏）、ai_agent 仓的 Gerrit 闭环运行日志；板上数据以提交 31782ca 的镜像实测（源码 mtime 与打包时刻一致）。

功能测试：各核心功能测试项与判据如下表。

表 1：功能测试结果

测试项	方法与判据（结果）
多核启动与烧录	all-app.bin 双镜像烧录后 NSH 启动，UART0 输入验证通过
跨核 RPMsg（AMP）	CPU1 独立 AP 镜像启动，RPMsg UART 控制台经 CP cu 可用，L2 打包可复现（IRQ79 门铃、共享 SRAM）
跨核网络（AP 上网）	usrsock over RPMsg：AP 侧 socket 经 CP 执行，ping www.baidu.com 域名解析 + 5/5、RTT 68-109 ms（evidence-20260920，镜像 31782ca）
SPI LCD	GC9D01 真机初始化、背光点亮、渲染（bit-bang 路径）；SPI1 硬件传输 CP 镜像链接与 L1 校验通过
Wi-Fi 扫描	chan_survey 打满 14 信道；scanu_confirm status=0、recv_cnt=104、result=32、单轮 1.67 s（约 30% 轮次扫描失败，重试即恢复）
关联与四次握手	ASSOCIATED → 4WAY_HANDSHAKE，PTK/KCK/KEK 推导成功，Sending EAPOL-Key 2/4
ARP / ICMP	ping 网关 4 发 4 收、0% 丢包（evidence-20260915 干净固件日志）
DHCP / DNS	renew wlan0 后 ifconfig 得到 inet addr:192.168.1.8；ping www.baidu.com 解析 110.242.74.102、10 发 10 收
DeskMate 协议	主机 gcc 单元测试加 Goldfish 模拟器通过（握手、决策闭锁、计费遥测、对话轮次）；acp-bridge npm 测试与 fake 回路通过（nonce 握手、参数脱敏、精确选项映射、重复决策抑制）
远程 Agent 控制	VelaClaw 控制台 rask 对话、ronce/rdeny 权限、rmodel 模型切换；桥端 session/request_permission 挂起直到设备 once/deny
Gerrit 编码闭环	QEMU 内 agent 经 MCP 完成读任务、写码、gcc 验证、amend 提交、推送、patchset 查

	询与播报（真实内网 Gerrit）

性能测试：实测数据如下表。另：IOB 缓冲从 196 调到 640 字节后收帧不再跨链（代价约 10.4 KB 堆），DHCP 随之完成握手。

表 2：性能实测数据

指标	实测数据
ping 网关（4 包）	4 收 0 丢，丢包率 0%
ping www.baidu.com（10 包）	10 收 0 丢，RTT 29-56 ms
PBKDF2（PMK 推导）	12.87 s（AP SMP 开启），关闭后降到 2.74 s（workaround；理论值 0.4 s 以内）
Wi-Fi 扫描单轮时长	1.67 s（recv_cnt=104 / result=32）
AP 侧 ping www.baidu.com（经 usrsock 跨核，5 包）	5 收 0 丢，RTT 68-109 ms；CP 侧对照 3/3、32-76 ms

可靠性与稳定性测试：① 扫描失败率：24 轮抓包统计，chan_survey 打满 14 信道则关联 15/18 成功，中途停止则 0/6，综合约 30%；属既有未解问题，重敲一次 connect 即恢复，已写入复盘文档作为兜底方案。② sched_unlock assert（serial.c:2064）：2/26 轮出现，ISR 回调 lockbal 探针四轮零输出，如实记录为未解。③ 桥的已知限制：观察到 MiMo 后端 bash 工具调用未发权限请求，桥只显示状态与工具结果、不伪造 Attention 提示，已记录在 acp-bridge README，不算权限控制成功。④ 未解项（跨 IOB 链丢帧机制、AP SMP 死锁、PBKDF2 余量）全部公开于复盘文档。

① 扫描失败率：24 轮抓包统计，chan_survey 打满 14 信道则关联 15/18 成功，中途停止则 0/6，综合约 30%；属既有未解问题，重敲一次 connect 即恢复，已写入复盘文档作为兜底方案。

② sched_unlock assert（serial.c:2064）：2/26 轮出现，ISR 回调 lockbal 探针四轮零输出，如实记录为未解。

③ 桥的已知限制：观察到 MiMo 后端 bash 工具调用未发权限请求，桥只显示状态与工具结果、

不伪造 Attention 提示，已记录在 acp-bridge README，不算权限控制成功。

④ 未解项（跨 IOB 链丢帧机制、AP SMP 死锁、PBKDF2 余量）全部公开于复盘文档。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	约 85%
使用的 AI 工具	Claude Code、MiMoCode、OpenCode、Codex（模型侧经 Xiaomi MiMo Token）
MCP 工具使用情况	设备端 ai_agent 的 MCP 客户端对接 remote_ctrl 服务器（rc_* 工具），驱动 Gerrit 编码闭环；MCP 客户端支持 x-user-token 鉴权接入 MCP 统一网关（已实现为 ai_agent 正式代码，随上游 PR #41 提交）
Skills 使用与新增情况	新增：bk7258-worktree-build（构建-打包-烧录闭环） 使用：官方 nuttx-driver-development、driver-code-reviewer、kconfig-tweak 等
Token 使用总量	Kiro 4W credits

补充：规模：3 名成员共 168 个 AI 会话（C-Luv9615 145、JUSTiceMaMo 17、suoyuan6666），自 2026-08-03 起覆盖开发全程，全部导出到仓内 logs/（manifest.json 加 JSONL）。效率提升：22 项根因的定位大量依赖 AI 做代码考古与假设推演（中断屏蔽契约、pbuf ABI 位错、EAPOL 分流义务），复盘文档与 Skill 由 AI 起草、人工校订；判据设计与"一次只改一个变量"的实验纪律由人主导。对 AI 幻觉的约束是判据先行：对闭源库行为的任何推测，必须落到可观测计数（netdev 统计、探针、ARP 表项）上才采信；固件一致性用源文件 mtime 对比打包时刻证明。对上游公共仓的改动按大赛流程以 PR 形式提交：packages_ai_agent 侧为 open-vela PR #41（dev-ai-contest-2026 分支，15 提交，含 MCP 网关鉴权、LVGL 卡片与通知、remote_ctrl 等），nuttx / vendor_openvela / apps_graphics_lvgl 的补丁同样走各仓 PR，底稿存 docs/zhangyi101/patches 供复现。

3.7 总结与展望

成果总结：

① openvela 在 BK7258 上从启动到联网（新硬件适配核心成果）：多核启动、双镜像打包烧录、驱动导入、显示点亮、Wi-Fi 全链路真机验证。

② openvela AI 能力的使用与扩展：ai_agent fork 上的远程 Agent 控制与 MCP 接入研发平台（Gerrit/Jira）；QEMU 上语音驱动的 Gerrit 编码闭环。

③ DeskMate Link 协议与双端实现（acp-bridge 与 deskmate），确定性权限模型经 fake 回路、Goldfish 模拟器与真机网络三层验证。

④ 工程沉淀：22 根因复盘文档、2 个自研 Skill、四批真机证据日志。

应用前景与商业价值：目标受众：需要 Wi-Fi SoC 选型的 openvela 硬件团队（智能音箱、桌面设备、IoT 网关），以及想把 AI 权限确认从 PC 屏幕挪到桌面实体设备的效率工具用户。商业模式与规模化：适配成果以 Apache 2.0 开源，设备厂商在 FreeRTOS 之外多一个 OS 选项；DeskMate 形态可延伸为带屏 AI 硬件，权限入口与远程交互是它与手机语音助手的差别。

不足与未来工作：未完成：Wi-Fi 适配不够彻底全面（扫描约 30% 失败的机制待查），稳定性不够强；AP SMP 需进一步排查；PBKDF2 性能优化（实测 2.74 s，理论应在 0.4 s 内）。下一步：deskmate 真机端到端联调，把图形与语音能力从 QEMU 向 BK7258 真机迁移。